

CPSC 375

Introduction to Data Science and Big Data Analytics

Dr. Kanika Sood

kasood@fullerton.edu

What we will cover today

- Joins
 - In Python with Pandas
- Tidy data

Join

- Merge two tables based on their common variables
- Two tables are called
 1. Left table
 2. Right table
- Steps to combine tables:
 1. Identify common variables
 2. Create a new table with variables from BOTH tables (including the common variables)
 3. Add values from every pair of rows where the values of the common variables match

Join

band_members (left table)

Name	Band
Mick	Stones
John	Beatles
Paul	Beatles

band_instruments (right table)

Name	Plays
John	guitar
Paul	bass
Keith	guitar

pd.merge()

Name	Band	Plays
John	Beatles	guitar
Paul	Beatles	bass

Join types

- Inner join – `pd.merge()`
 - retain (only) observations where this is a match in **both** datasets (left and right)
- Left join – `pd.merge(..., how='left')`
 - Keep all rows in **left-hand** dataset, add values from right-hand dataset where there is a match.
 - For non-matching observations, fill in NA
- Right join – `pd.merge(..., how='right')`
 - Keep all rows in **right-hand** dataset, add values from left-hand dataset where there is a match.
 - For non-matching observations, fill in NA
- Full Join / Outer Join – `pd.merge(..., how='outer')`
 - Combine observations from **both** datasets based on matching key **and** fill in NA for non-matches.

Join types

band_members (left table)

Name	Band
Mick	Stones
John	Beatles
Paul	Beatles

band_instruments (right table)

Name	Plays
John	guitar
Paul	bass
Keith	guitar

- `pd.merge(band_members, band_instruments)`
- `pd.merge(band_members, band_instruments, how='left')`
- `pd.merge(band_members, band_instruments, how='right')`
- `pd.merge(band_members, band_instruments, how='outer')`

Join types: left join

band_members (left table)

Name	Band
Mick	Stones
John	Beatles
Paul	Beatles

band_instruments (right table)

Name	Plays
John	guitar
Paul	bass
Keith	guitar

`pd.merge(..., how='left')`

Name	Band	Plays
Mick	Stones	<i>NaN</i>
John	Beatles	guitar
Paul	Beatles	bass

Join types

band_members (left table)

Name	Band
Mick	Stones
John	Beatles
Paul	Beatles

band_instruments_4rows (right table)

Name	Plays
John	guitar
Paul	bass
Keith	guitar
Paul	vocals

If there are multiple matches, ALL combinations of left, right are returned

Join types

band_members (left table)

Name	Band
Mick	Stones
John	Beatles
Paul	Beatles

band_instruments_4rows (right table)

Name	Plays
John	guitar
Paul	bass
Keith	guitar
Paul	vocals

If there are multiple matches, ALL combinations of left, right are returned

Name	Band	Plays
John	Beatles	guitar
Paul	Beatles	bass
Paul	Beatles	vocals

```
pd.merge(...,  
how='inner')
```

Class work

- [soodk6/cpsc375-classwork](#)
–wrangling-tidy.ipynb – Part 1

Join types

band_members_4rows (left table)

Name	Band
Mick	Stones
John	Beatles
Paul	Beatles
Paul	Beatles

band_instruments_4rows (right table)

Name	Plays
John	guitar
Paul	bass
Keith	guitar
Paul	vocals

Inner join

Name	Band	Plays
John	Beatles	guitar
Paul	Beatles	bass
Paul	Beatles	vocals
?	?	?
?	?	?

Join types

band_members (left table)

Name	Band	Plays
Mick	Stones	Vocals
John	Beatles	Lead
Paul	Beatles	bass

band_instruments (right table)

Name	Plays
John	guitar
Paul	bass
Keith	guitar

Name	Band	Plays
Paul	Beatles	bass

Filter join types

- Create a new table with variables from only LEFT table (including the common variables)
- **Semi-join**
 - return all rows from LEFT where there are matching values in RIGHT, keeping just columns from LEFT

```
(  
    pd.merge(left, right, how='left', indicator=True)  
    .query("_merge == 'both'")  
    .filter(left.columns)  
)
```

Filter join types

- Create a new table with variables from only LEFT table (including the common variables)
- **Anti-join**
 - return all rows from LEFT where there are **not** matching values in RIGHT, keeping just columns from LEFT.

```
(  
    pd.merge(left, right, how='left', indicator=True)  
    .query("_merge == 'left_only'")  
    .filter(left.columns)  
)
```

Special values to watch out for

- NaN:
 - Missing, or
 - Result of an invalid computation, e.g., $\log(-1)$
 - `skipna=True` default in many functions
- None:
 - Often used when something is undefined
- `np.inf`: infinite
- Depending on desired result and join columns, may need to drop duplicates
- **Warnings:**
 - If Python mentions a warning in data wrangling, make sure you've handled it or know its origin.

Data structure semantics

- Most data wrangling can be accomplished using DataFrame objects.
 - Columns are typically labeled (and represent variables)
 - Rows have indexes
 - Can be labeled, although often not necessary
- Datasets contain **values**: numbers, strings, etc.
- A value belongs to a **variable** and an **observation**.
- A variable contains all values measuring an attribute (e.g., height) across units (e.g., people)
- An observation contains all values measured on the same unit across attributes.

Wickham 2016 tidyR tutorial

Data structure semantics

country	year	cases	population
Afghanistan	1999	7745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	128042583

variables

country	year	cases	population
Afghanistan	1999	7745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	128042583

observations

country	year	cases	population
Afghanistan	999	7745	19987071
Afghanistan	000	2666	20595360
Brazil	999	37737	172006362
Brazil	000	80488	174504898
China	999	212258	1272915272
China	000	213766	128042583

values

<http://garrettgman.github.io/tidying/>

Tidy data

1. Each variable forms a column
2. Each observation forms a row
3. Each type of observational unit (e.g., persons, schools, counties) forms a table.

Variables that are part of the design (participant number, experimental condition, county ID, etc.), or that may be key categorical moderators, should typically be placed first (to the left), and measured variables thereafter.

Wickham 2014

Why Tidy Data?

- Consistency:
 - If you have a consistent data structure, it's **easier** to learn the tools that work with it because they have an underlying uniformity
- Column-based format
 - allows Pandas and NumPy's vectorized nature to shine.
 - Most Pandas and NumPy methods work with vectors of values.
 - Python sequence methods overloaded to work with vectors of values

Is this a good data way to store data?

CWID	CPSC120	CPSC121	CPSC131	MATH338
1234567	3.0	3.3	3.0	3.0
2356902	4.0	3.0	4.0	NA
5005667	NA	2.0	3.7	3.3

A better way to store data

CWID	Course	Grade
1234567	CPSC120	3.0
1234567	CPSC121	3.3
1234567	CPSC131	3.0
1234567	MATH338	3.0
2356902	CPSC120	4.0
2356902	CPSC121	3.0
2356902	CPSC131	4.0
5005667	CPSC121	2.0
5005667	CPSC131	3.7
5005667	MATH338	3.3

Mess #1: Column headers are values, not variable names

Messy

religion	<\$10k	\$10–20k	\$20–30k	\$30–40k	\$40–50k	\$50–75k
Agnostic	27	34	60	81	76	137
Atheist	12	27	37	52	35	70
Buddhist	27	21	30	34	33	58
Catholic	418	617	732	670	638	1116
Don't know/refused	15	14	15	11	10	35
Evangelical Prot	575	869	1064	982	881	1486
Hindu	1	9	7	9	11	34
Historically Black Prot	228	244	236	238	197	223
Jehovah's Witness	20	27	24	24	21	30
Jewish	19	19	25	25	30	95

Tidy

religion	income	freq
Agnostic	<\$10k	27
Agnostic	\$10–20k	34
Agnostic	\$20–30k	60
Agnostic	\$30–40k	81
Agnostic	\$40–50k	76
Agnostic	\$50–75k	137
Agnostic	\$75–100k	122
Agnostic	\$100–150k	109
Agnostic	>150k	84
Agnostic	Don't know/refused	96

Mess #1: Column headers are values, not variable names

Messy

Tidy: ?

	country	1999	2000
0	Afghanistan	745	2666
1	Brazil	37737	80488
2	China	212258	213766

Data tidying verbs: .melt()

- .melt():
 - Increases the number of rows and decreases the number of columns
 - “Lengthens” data
- Parameters to .melt():
 - **id_vars**: The names of “identifier” columns
 - columns that are part of the design or are key categorical moderators
 - **value_vars**: The names of columns to “pivot” into longer format
 - “old” columns (default is all columns other than id_vars)
 - **var_name**: Name of the new column to create from the names of the old columns
 - **value_name**: Name of the new column to create from the values of the old columns

Mess #1: Column headers are values, not variable names

Messy

religion	<\$10k	\$10–20k	\$20–30k	\$30–40k	\$40–50k	\$50–75k
Agnostic	27	34	60	81	76	137
Atheist	12	27	37	52	35	70
Buddhist	27	21	30	34	33	58
Catholic	418	617	732	670	638	1116
Don't know/refused	15	14	15	11	10	35
Evangelical Prot	575	869	1064	982	881	1486
Hindu	1	9	7	9	11	34
Historically Black Prot	228	244	236	238	197	223
Jehovah's Witness	20	27	24	24	21	30
Jewish	19	19	25	25	30	95

Tidy

religion	income	freq
Agnostic	<\$10k	27
Agnostic	\$10–20k	34
Agnostic	\$20–30k	60
Agnostic	\$30–40k	81
Agnostic	\$40–50k	76
Agnostic	\$50–75k	137
Agnostic	\$75–100k	122
Agnostic	\$100–150k	109
Agnostic	>150k	84
Agnostic	Don't know/refused	96



```
relig_income.melt(  
  id_vars=['religion'],  
  value_vars=relig_income.columns.drop(['religion']),  
  var_name='income',  
  value_name='freq')
```

Mess #1: Column headers are values, not variable names

Messy (table4a)

	country	1999	2000
0	Afghanistan	745	2666
1	Brazil	37737	80488
2	China	212258	213766



Tidy:

	country	year	cases
0	Afghanistan	1999	745
1	Brazil	1999	37737
2	China	1999	212258
3	Afghanistan	2000	2666
4	Brazil	2000	80488
5	China	2000	213766

```
table4a.melt(["country"], var_name="year", value_name="cases")
```

.melt()

- By default, NA values are not dropped
- Can drop by adding
`.dropna(subset=[value_name])` :

```
grades.melt( ..., var_name=..., value_name=...) \  
  .dropna(subset=[...])
```

Class work

- [soodk6/cpsc375-classwork](#)
– wrangling-tidy.ipynb – Part 2

CWID	NAME	CPSC120	CPSC121	CPSC131	MATH338
1234567	Mary	3.0	3.3	3.0	3.0
2356902	Lee	4.0	3.0	4.0	NA
5005667	Elena	NA	2.0	3.7	3.3

Tidy Data

“Happy families are all alike; every unhappy family is unhappy in its own way.” — Leo Tolstoy

“Tidy datasets are all alike, but every messy dataset is messy in its own way.” — Hadley Wickham

Mess #2: One observation in multiple rows

Messy

<u>country</u>	<u>year</u>	<u>type</u>	<u>count</u>
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362



Tidy

<u>country</u>	<u>year</u>	<u>cases</u>
<u>population</u>		
Afghanistan	1999	745
19987071		
Afghanistan	2000	2666
20595360		
Brazil	1999	37737
172006362		
Brazil	2000	80488
174504898		
China	1999	212258
1272915272		
China	2000	213766
1280428583		

Mess #2: One observation in multiple rows

Messy

<u>country</u>	<u>year</u>	<u>type</u>	<u>count</u>
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362



Tidy

<u>country</u>	<u>year</u>	<u>cases</u>
<u>population</u>		
Afghanistan	1999	745
19987071		
Afghanistan	2000	2666
20595360		
Brazil	1999	37737
172006362		
Brazil	2000	80488
174504898		
China	1999	212258
1272915272		
China	2000	213766
1280428583		

Data tidying verb: pivot

.pivot(): used when an observation is scattered across **rows**

Mess #2: One observation in multiple rows

country	year	type	count
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898
China	1999	cases	212258
China	1999	population	1272915272
China	2000	cases	213766
China	2000	population	1280428583

table2

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

```
table2.pivot(  
    index=['country', 'year'], columns='type', values='count'  
).reset_index().rename_axis(None, axis=1)
```

Classwork

- [soodk6/cpsc375-classwork](#)
– wrangling-tidy.ipynb – Part 3

Mess #3: Multiple variables stored in one column

Messy (table3)

<u>country</u>	<u>year</u>	<u>rate</u>
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583



Tidy

<u>country</u>	<u>year</u>	<u>cases</u>	<u>population</u>
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Data tidying verb: `split()`

Separate: split rate column

Default: use whitespace as separator

```
table3[['cases', 'population']] = table3.rate.str.split('/', expand=True)
table3.drop('rate', axis=1)
```

Mess #3: Multiple variables stored in one column

Messy (table3)

<u>country</u>	<u>year</u>	<u>rate</u>
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583



Tidy

<u>country</u>	<u>year</u>	<u>cases</u>	<u>population</u>
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Data tidying verb: `split()`

Split rate column:

```
table3.assign(
```

```
    temp=lambda df: df['rate'].str.split('/'),
```

```
    cases=lambda df: df.temp.str[0].astype('int'),
```

```
    population=lambda df: df.temp.str[1].astype('int')
```

```
).drop(columns=['temp']))
```

Class work

- [soodk6/cpsc375-classwork](#)
–wrangling-tidy.ipynb – Part 4

Class work

- [soodk6/cpsc375-classwork](#)
– wrangling-tidy.ipynb – Part 5

Announcements

- HW3
 - Due soon
- HW4
 - Due soon

Thank you

Announcement

- MidTerm: Week 09
 - On Canvas
 - In class
 - Open notes, open book

Mentimeter



DSML Flyer

